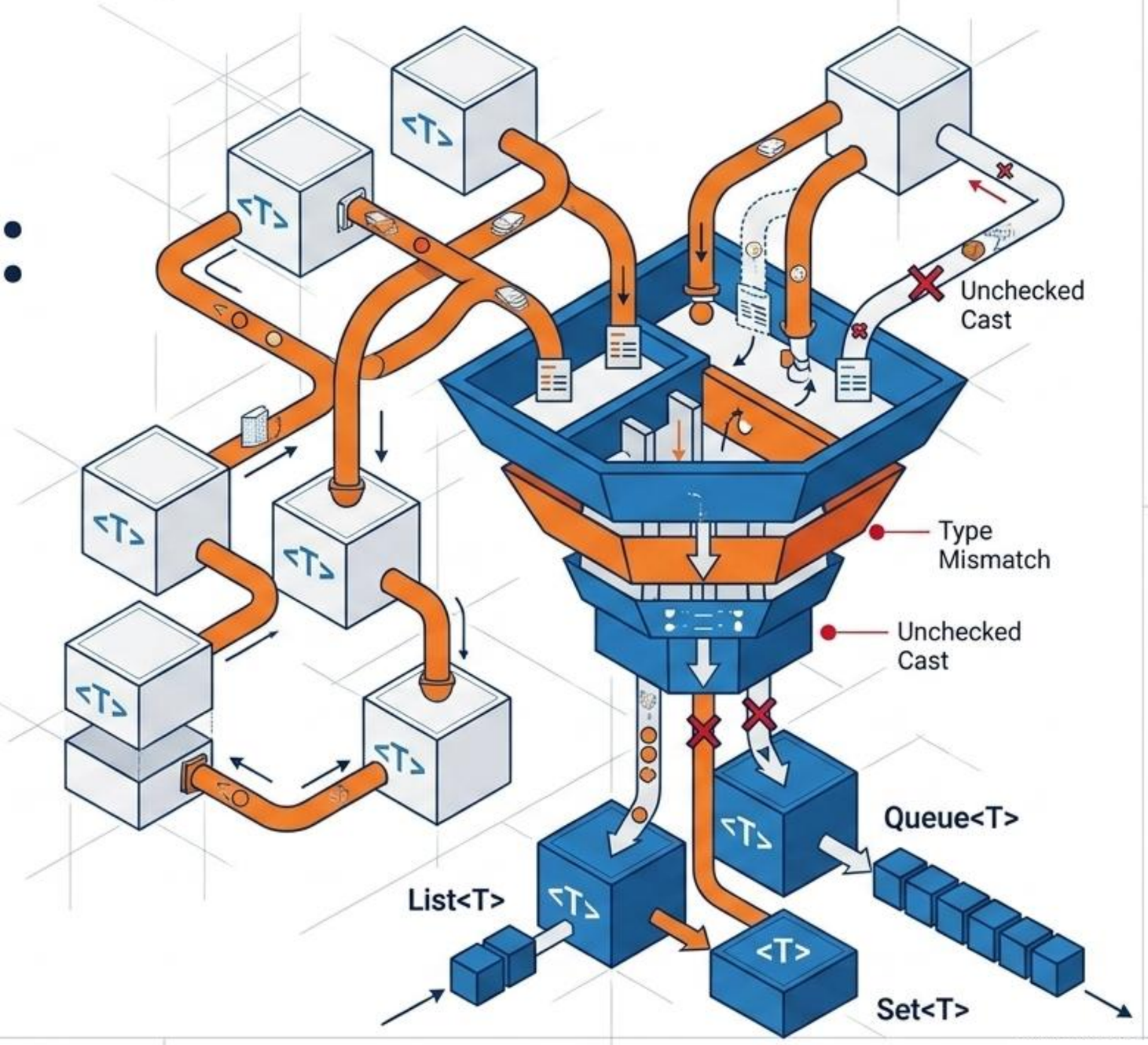
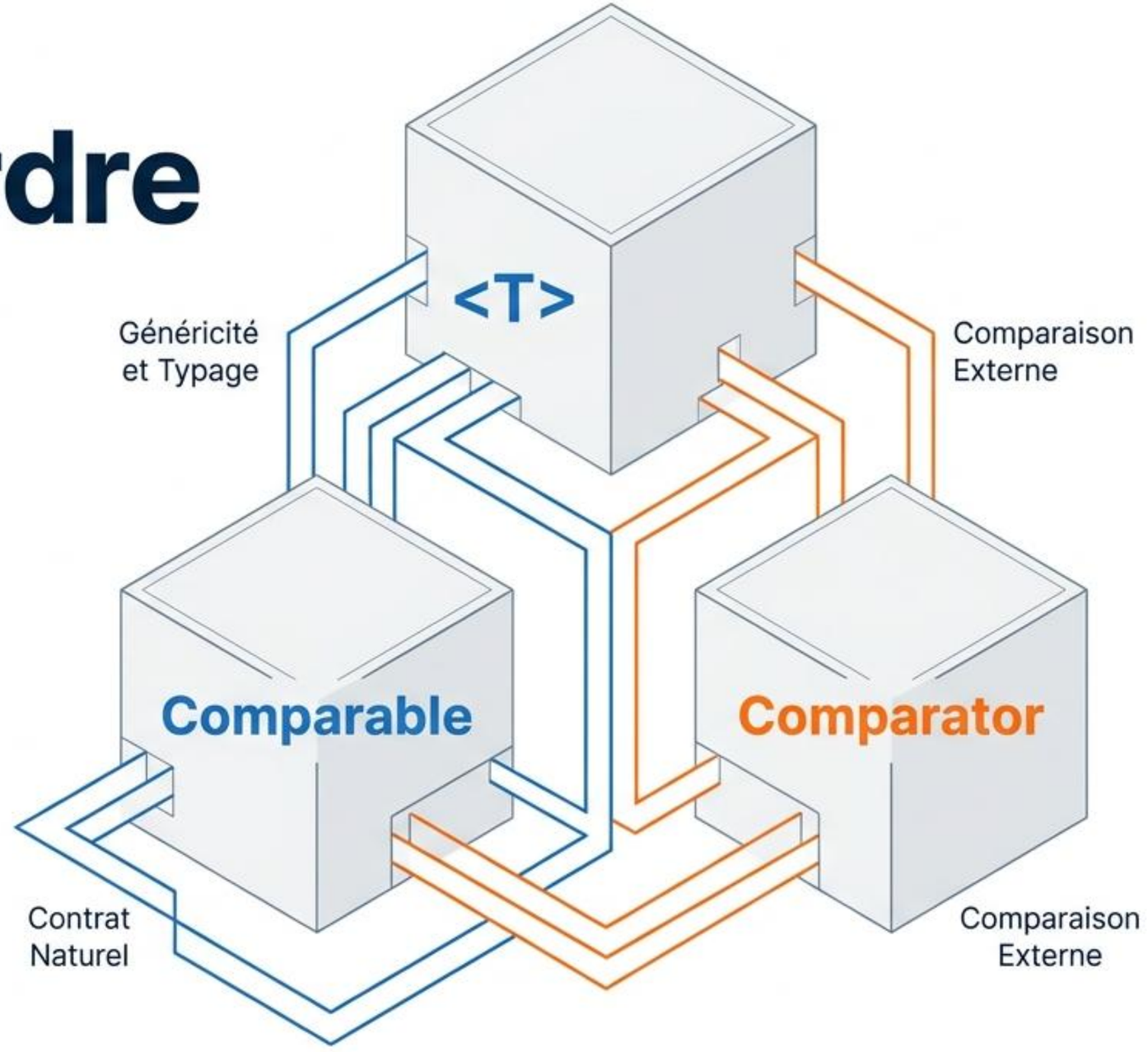
Architecture et Typage en Java : Des Génériques aux Collections

Un guide visuel de référence pour structurer, ordonner et sécuriser vos données.



Maîtriser l'Ordre et le Typage



Le Problème : Un Monde sans Filet de Sécurité

Synthèse : Avant Java 5, le compilateur ignorait le contenu réel des collections. L'erreur humaine n'était découverte qu'à l'exécution, provoquant des crashs inattendus.



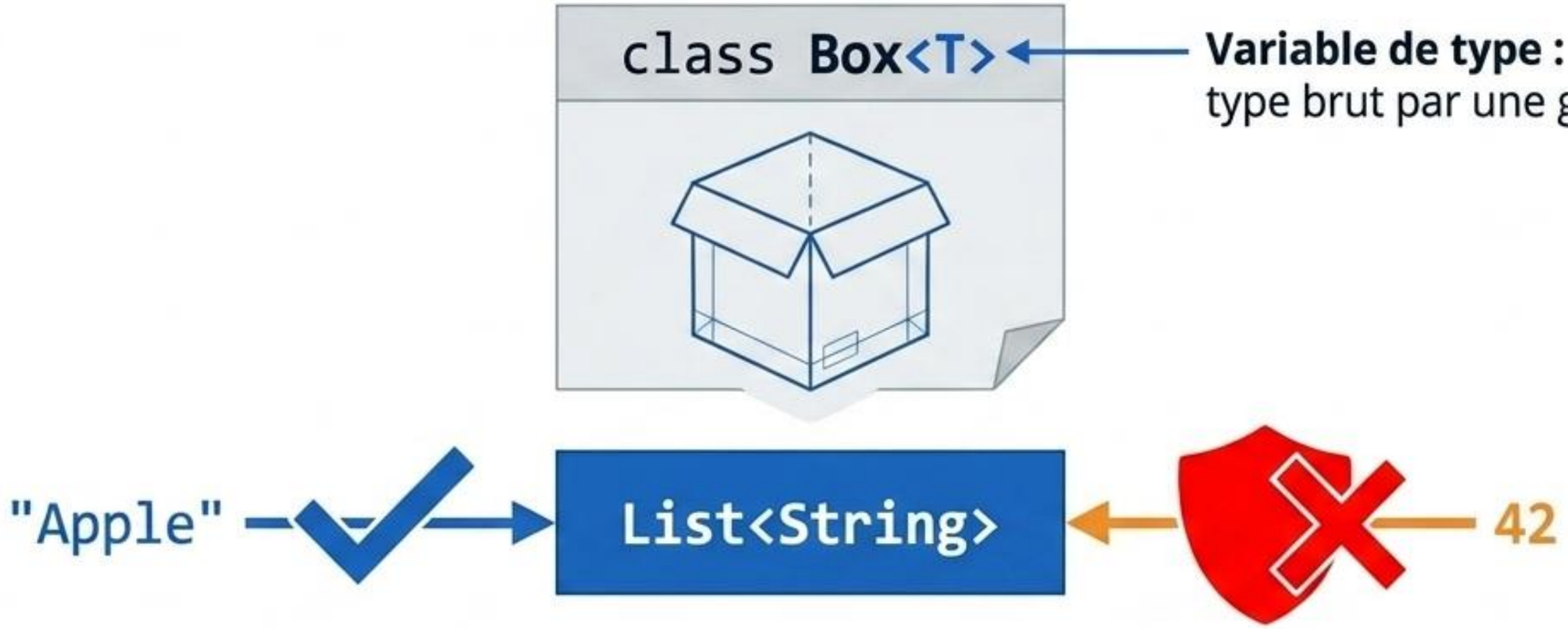

```
List words = new ArrayList();
words.add("Hello");
words.add(new Car());

String s = (String) words.get(1); // CAST DANGEREUX
```

Runtime Error: ClassCastException

Génériques : Paramétrer les Types

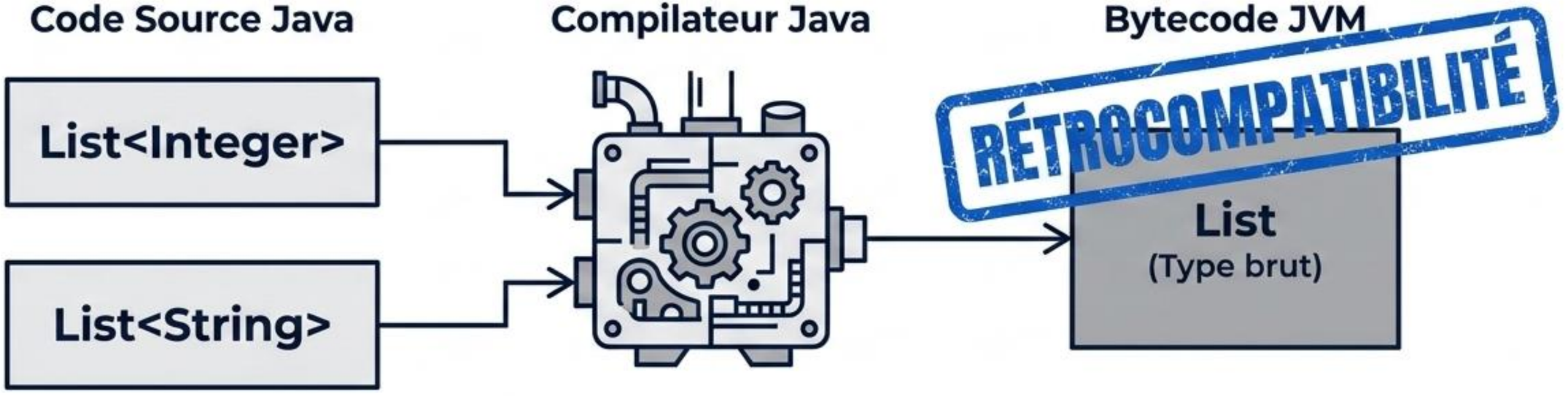
Synthèse : Les Génériques déplacent les vérifications de type du Runtime (Exécution) vers le Compile-time (Compilation). Les casts manuels disparaissent.




```
List<String> list = new ArrayList<>();
list.add("Apple"); // ✔
list.add(42); // ✘ Compile-time error
```

L'Illusion du Bytecode (Type Erasure)

L'Effacement de Type (Type Erasure) supprime les paramètres génériques après la compilation. La JVM exécute un **bytecode identique au code** pré-Java 5, garantissant **la compatibilité avec les anciens systèmes.**



Ordonner des Objets Complexes

Nom	Âge	Rang
Alice	24 ans	2
Bob	30 ans	1
Charlie	22 ans	3

```
List<Player> players =
    Arrays.asList(alice, bob, charlie);

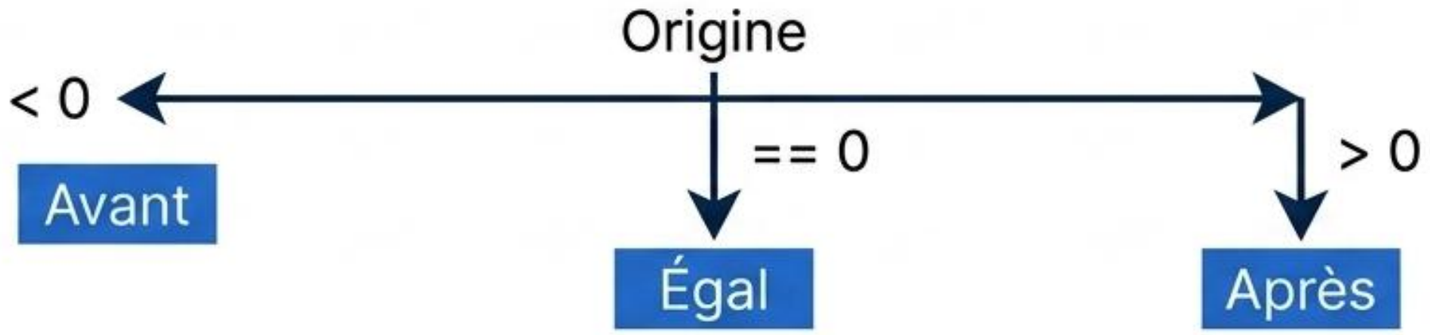
Collections.sort(players); // ❌ ERREUR
```

Java sait trier des entiers (1, 2, 3) ou des lettres (A, B, C).
Mais comment trier un objet Player ? Par âge ? Par rang ?
Le compilateur exige une stratégie de comparaison.

Comparable : Définir l'Ordre Naturel

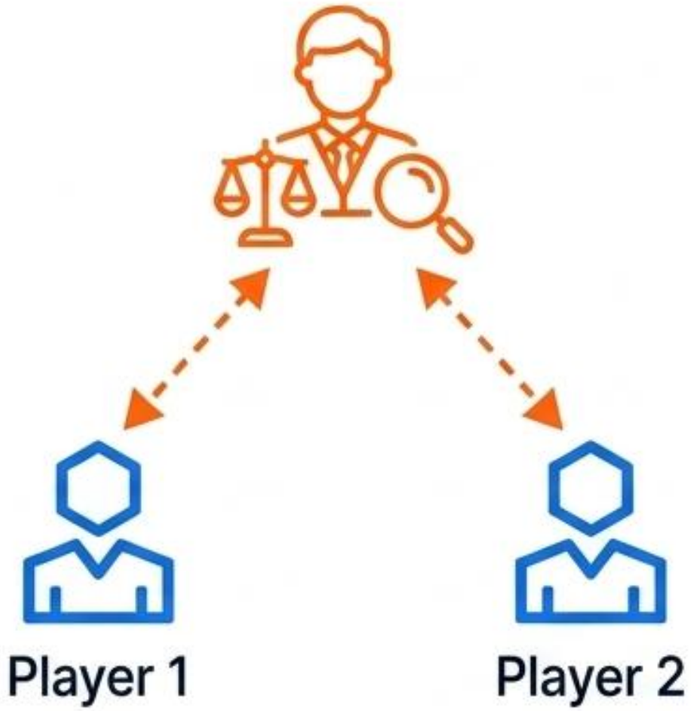
Synthesis: Implémenter Comparable dicte le comportement de tri par défaut depuis l'intérieur de la classe. C'est l'ordre "naturel".

```java
public class Player implements Comparable<Player> {
    public int compareTo(Player other) {
        return Integer.compare(this.ranking, other.ranking);
    }
}
```



Comparator : L'Ordre Sur-Mesure

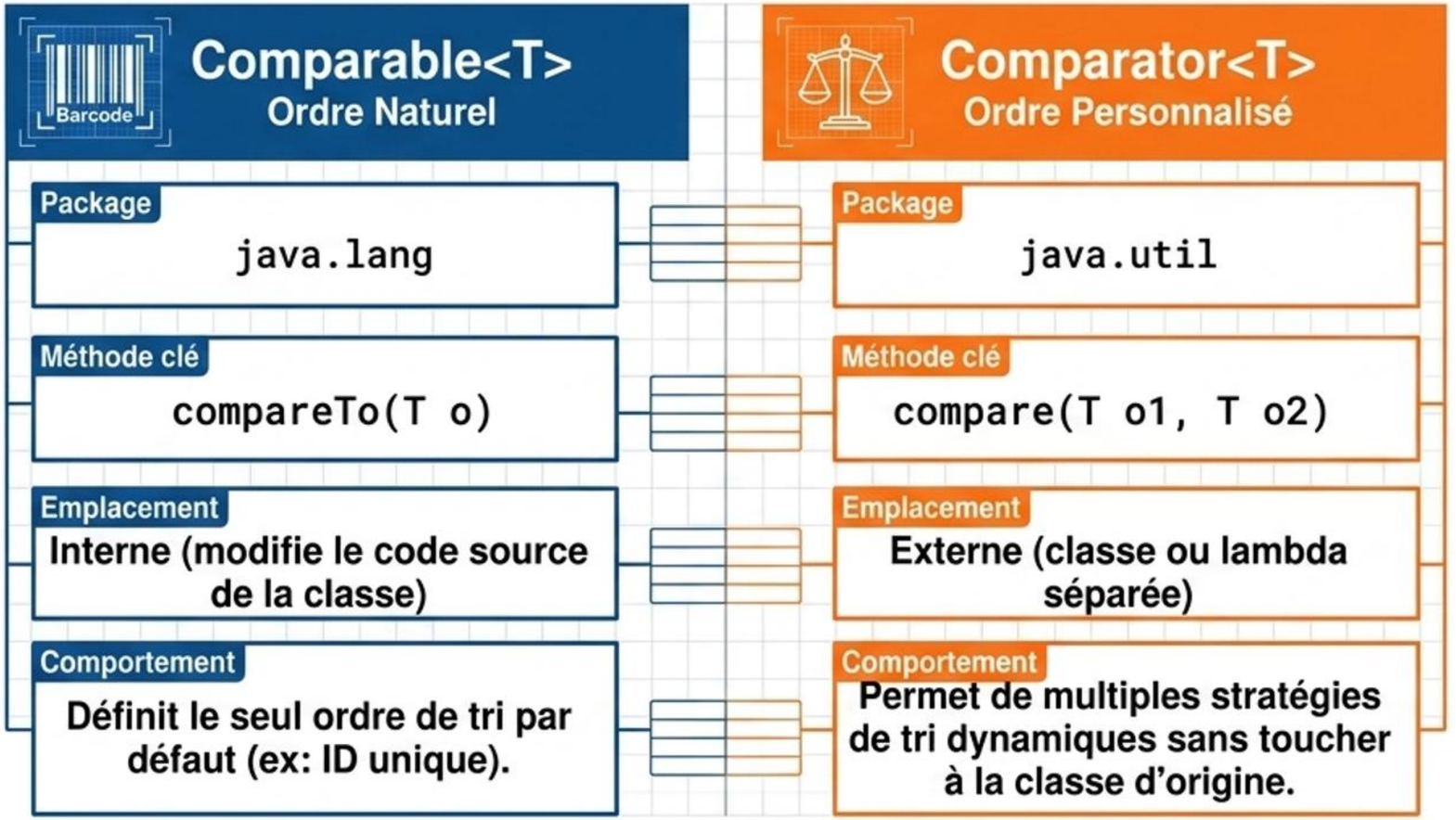
L'interface **Comparator** permet de définir de multiples stratégies de tri de **manière extrinsèque**, sans modifier le code source de la **classe originale**. Idéal pour des affichages dynamiques.




```
class PlayerAgeComparator implements Comparator<Player> {
    public int compare(Player p1, Player p2) {
        return Integer.compare(p1.age, p2.age);
    }
}

// Utilisation :
Collections.sort(players, new PlayerAgeComparator());
```

Matrice de Décision : Comparable vs Comparator



Utilisez Comparable pour l'ordre métier fondamental.
Utilisez des Comparator pour les besoins dynamiques des interfaces utilisateur.

Java 8 : La Révolution des Comparators

Synthèse : Grâce aux méthodes de fabrique statiques génériques (comparing) et aux références de méthodes, la création de stratégies de tri devient totalement déclarative.

Avant (Verbeux)

```
Comparator<Player> byAge = new Comparator<Player>() {
    public int compare(Player p1, Player p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
};
```



Après (Déclaratif)

```
Comparator<Player> byAge = Comparator.comparing(Player::getAge);
```

Combiner les Critères avec Fluidité

Les Default Methods des interfaces Java 8 permettent de chaîner les comparateurs. Une logique complexe s'écrit de manière aussi lisible que du langage naturel.




```
players.sort(Comparator.comparing(Player::getName)
                       .reversed()
                       .thenComparing(Player::getAge));
```

Limiter les Génériques (Type Bounds)

Synthesis :
Les limites (Bounds) imposent un contrat strict au moment de la compilation.
Le compilateur refusera d'exécuter `findMax` sur une liste d'objets non ordonnables.

```
public static <T extends Comparable<T>> T findMax(List<T> list)
```

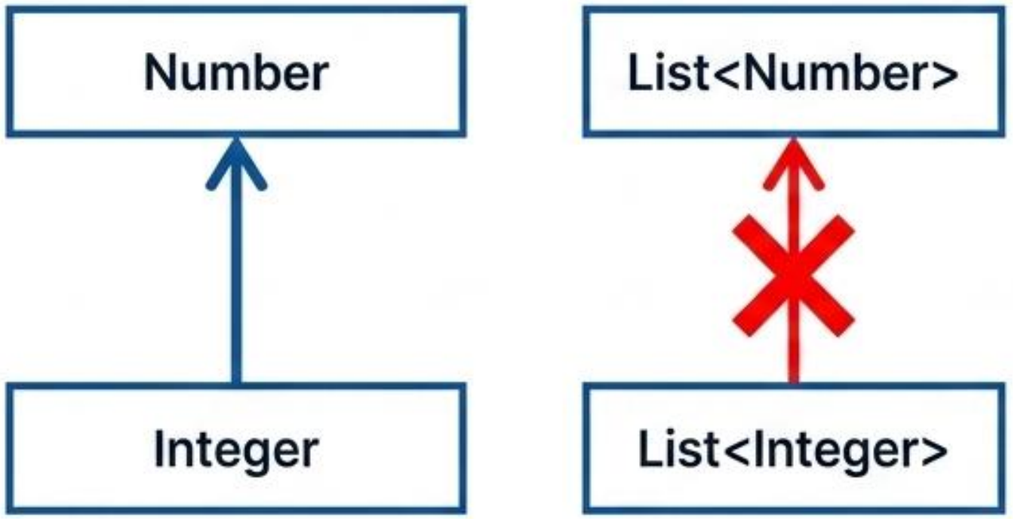


1. **Objectif** : Trouver le maximum dans une liste générique.
2. **Contrainte** : Le compilateur doit s'assurer que les éléments peuvent être comparés entre eux.
3. **Solution** : Le mot-clé `extends` oblige le type `T` à implémenter l'interface de comparaison.

Gérer l'Héritage : Les Jokers (Wildcards)

Les Wildcards (?) assouplissent les règles strictes des génériques pour permettre le polymorphisme, tout en gardant un contrôle absolu sur le typage.

Le Piège



Problème d'invariance

La Solution

```
// Le Joker (?) rend la méthode flexible
public double sum(List<? extends Number> list) {
    ...
}
```

✓ List<? extends Number> accepte désormais List<Integer> et List<Double> (Limite supérieure).

Pièges Courants et Anti-Patterns

L'Astuce de la Soustraction

- **Erreur :**

  `return a - b;` dans un `Comparator`.

- **Risque :**

  Integer Overflow si a est très positif et b très négatif.

- **Solution :**

  `Utiliser la sécurité de Integer.compare(a, b).`

Le Piège des Primitives

- **Erreur :**

  `Essayer de déclarer List<int>.`

- **Risque :**

  Les types paramétrés exigent des objets à cause du Type Erasure.

- **Solution :**

  Utiliser les classes enveloppes (`Integer`) ou `Comparator.comparingInt()`.

Types Bruts (Raw Types)

- **Erreur :**

  `Déclarer List myData = new ArrayList();`

- **Risque :**

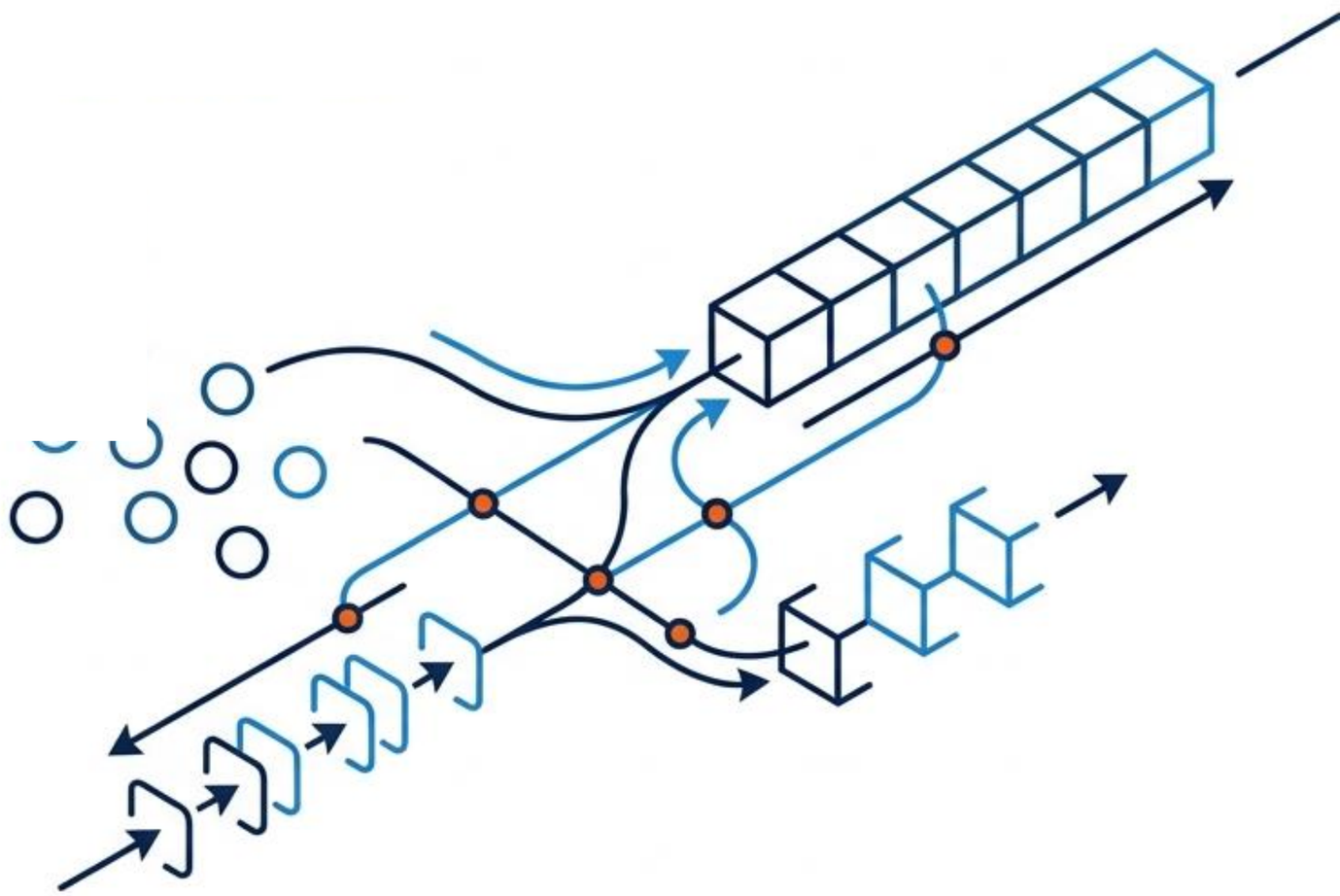
  Perte totale du filet de sécurité du compilateur. Risque de ClassCastException.

- **Solution :**

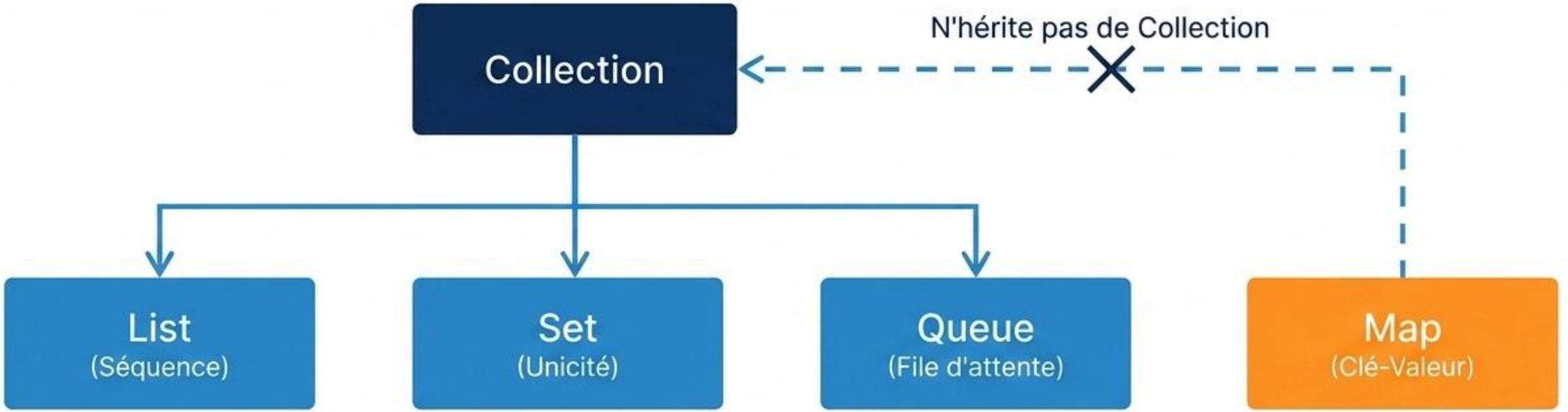
  Ne jamais omettre les chevrons `<>`.

Maîtriser les Collections

List, Set et Queue

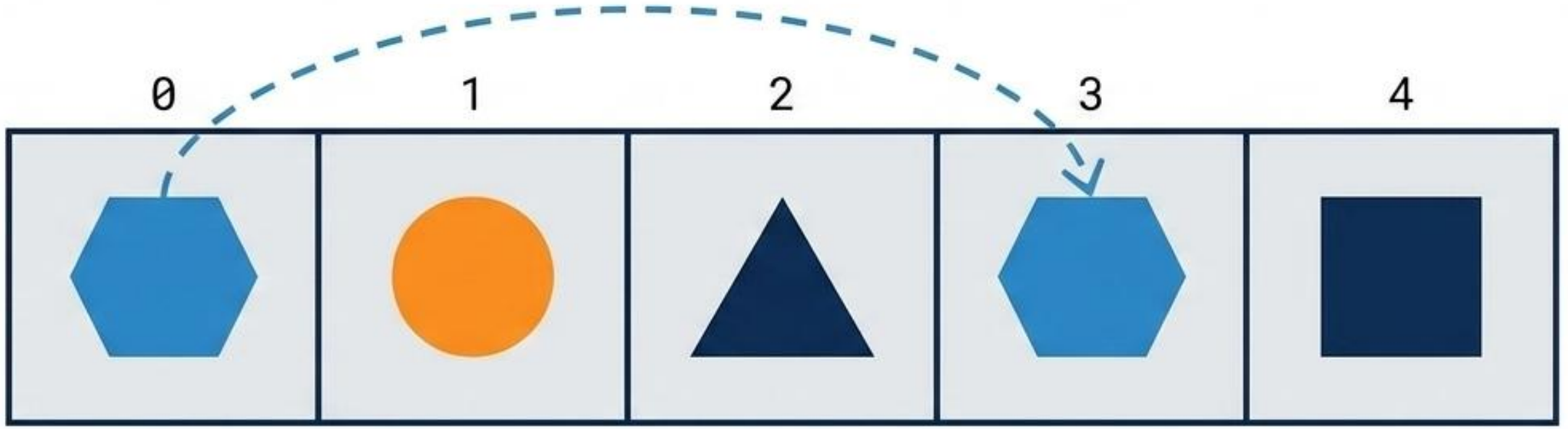


L'Architecture Globale des Collections



Toutes ces structures partagent des méthodes communes (add, remove, size, isEmpty), mais diffèrent par la manière dont elles organisent les données.

List Représente la Séquence Ordonnée



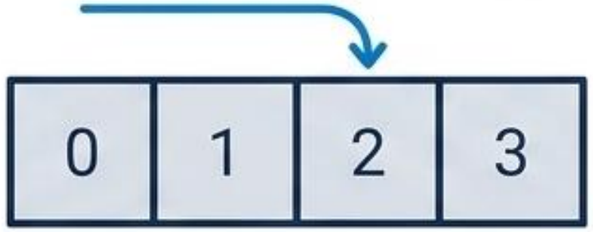
Ordre garanti : Les éléments conservent l'ordre exact d'insertion.

Accès par index : Manipulation ultra-précise (`get(index)`, `add(index, element)`).

Doublons autorisés : Un même objet peut apparaître plusieurs fois.

Comparatif des Implémentations de List

ArrayList



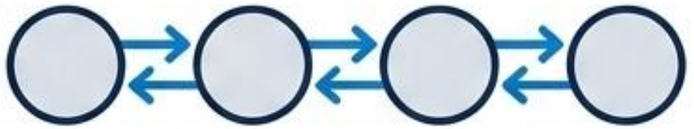
PROS

O(1) Accès direct par index. Idéal pour la lecture.

CONS

Insertions lentes au milieu (nécessite un décalage via System.arraycopy())

LinkedList



PROS

Insertions/suppressions ultra-rapides aux extrémités.

CONS

Parcours O(n) obligatoire, chasse aux pointeurs lente.

Vector



Thread-safe mais classe legacy obsolète. Lourdes pénalités de performance.

List en Pratique : Accès vs Manipulation

ArrayList (Le standard)

```
List<String> readHeavy =
new ArrayList<>();

readHeavy.add("Data");
String val = readHeavy.get(0);

// O(1) - Excellent pour les lectures
fréquentes
```

LinkedList (La manipulation d'extrémités)

```
LinkedList<String> edgeHeavy =
new LinkedList<>();

edgeHeavy.addFirst("Head");
edgeHeavy.addLast("Tail");

// O(1) - Idéal comme file d'attente
ou pile
```

Set Agit Comme un Filtre d’Unicité

Zéro doublon : Garantit l'unicité absolue. add(e) retourne false si l'élément existe déjà.

Pas d’index : L'accès se fait uniquement par recherche (contains()) ou itération, jamais par position.



Comparatif des Implémentations de Set

HashSet



Performance : O(1)

Ordre : Aucun (Aléatoire)

Null : 1 autorisé

LinkedHashSet

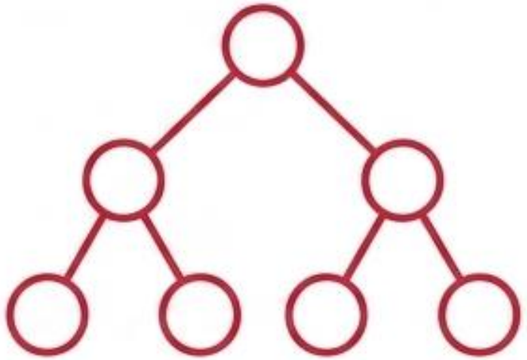


Performance : O(1) (léger surcoût)

Ordre : Préserve l'ordre d'insertion

Null : 1 autorisé

TreeSet



Performance : O(log n)

Ordre : Tri naturel

Null : Interdit (Exception fatale)

Set en Pratique : Déduplication et Tri

Liste avec doublons

```
List<String> emails =
  List.of("a@a.com",
          "b@b.com",
          "a@a.com");
```



Astuce : Déduplication en 1 ligne

```
Set<String> uniqueEmails =
    new HashSet<>(emails);
```

TreeSet trie automatiquement

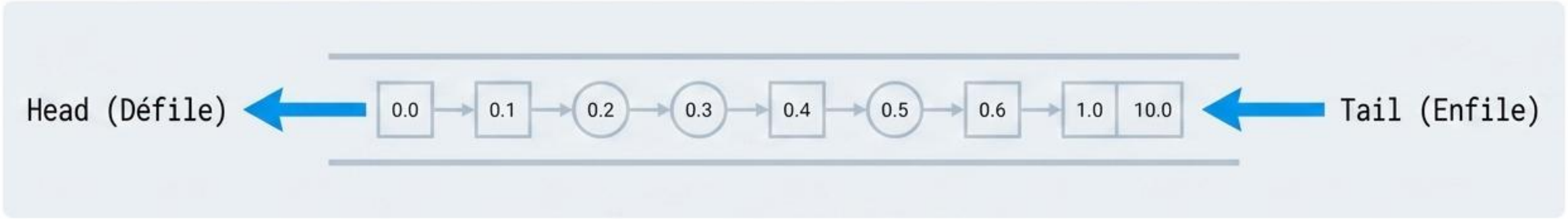
```
Set<String> sortedNames =
  new TreeSet<>();
sortedNames.add("Zoe");
sortedNames.add("Alice");
```



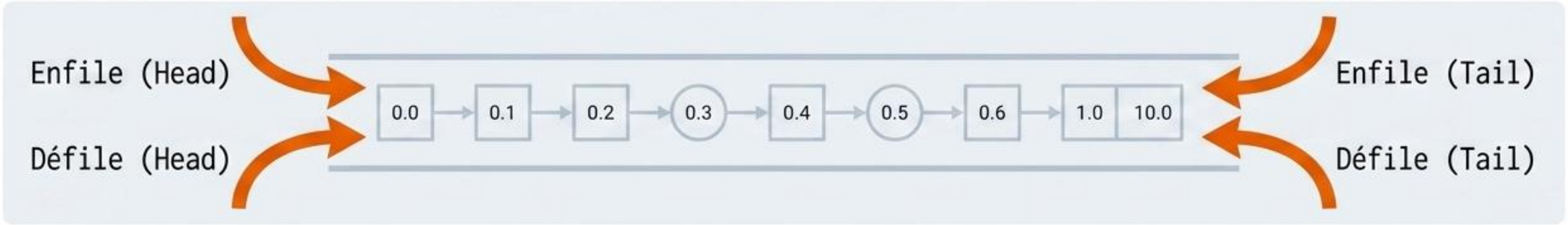
```
[Alice, Zoe]
```

Queue et Deque Gèrent le Flux de Données

Queue (FIFO)



Deque (Double-Ended Queue)



Peut agir comme FIFO (File) ou LIFO (Pile).

La Gestion des Erreurs dans l'API Queue

Lève une Exception

Plante si la file est vide/pleine.
Utilisé pour les flux stricts.

```
add(e) // Insérer
remove()  // Extraire
element() // Inspecter
```

Retourne une Valeur spéciale

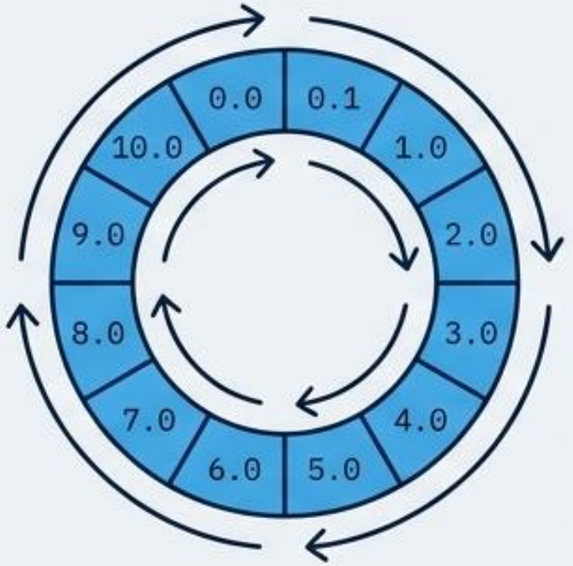
Retourne false ou null gracieusement.
Pour le code robuste et concurrent.

```
offer(e) // Insérer
poll()   // Extraire
peek()   // Inspecter
```

Comparatif des Implémentations de Queue

ArrayDeque

Le Standard Moderne

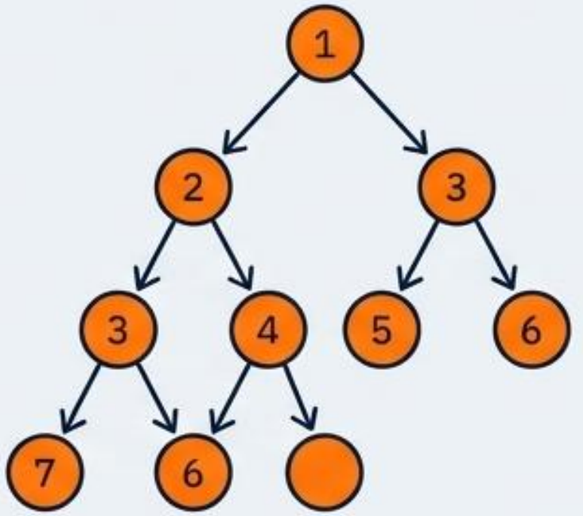


- Extrêmement rapide, faible empreinte mémoire.

⚠ **Remplace la classe obsolète Stack pour les piles (LIFO).**

PriorityQueue

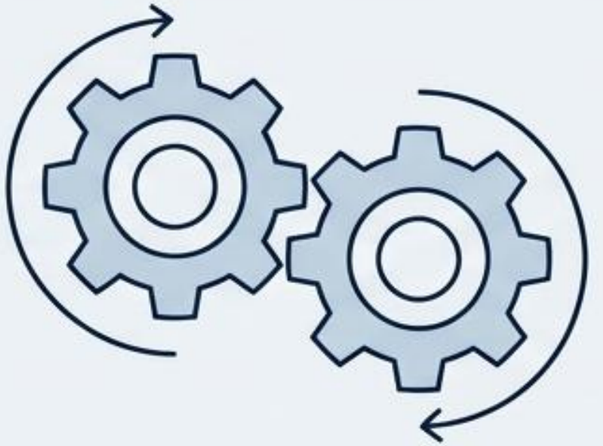
Le Trieur Automatique



- N'est pas FIFO.
- Le plus petit élément sort toujours en premier.
- Valeurs null strictement interdites.

Queues Concurrentes

Multi-threading



- BlockingQueue (ex: ArrayBlockingQueue) idéal pour les architectures Producteur/Consommateur.

Queue en Pratique : Piles et Priorités

LIFO (Pile) avec ArrayDeque

```
Deque<String> pile = new ArrayDeque<>();

pile.push("Tâche 1");
pile.push("Tâche 2");

pile.pop();
pile.pop(); ⟶ // Retourne "Tâche 2"
```

Ordre magique avec PriorityQueue

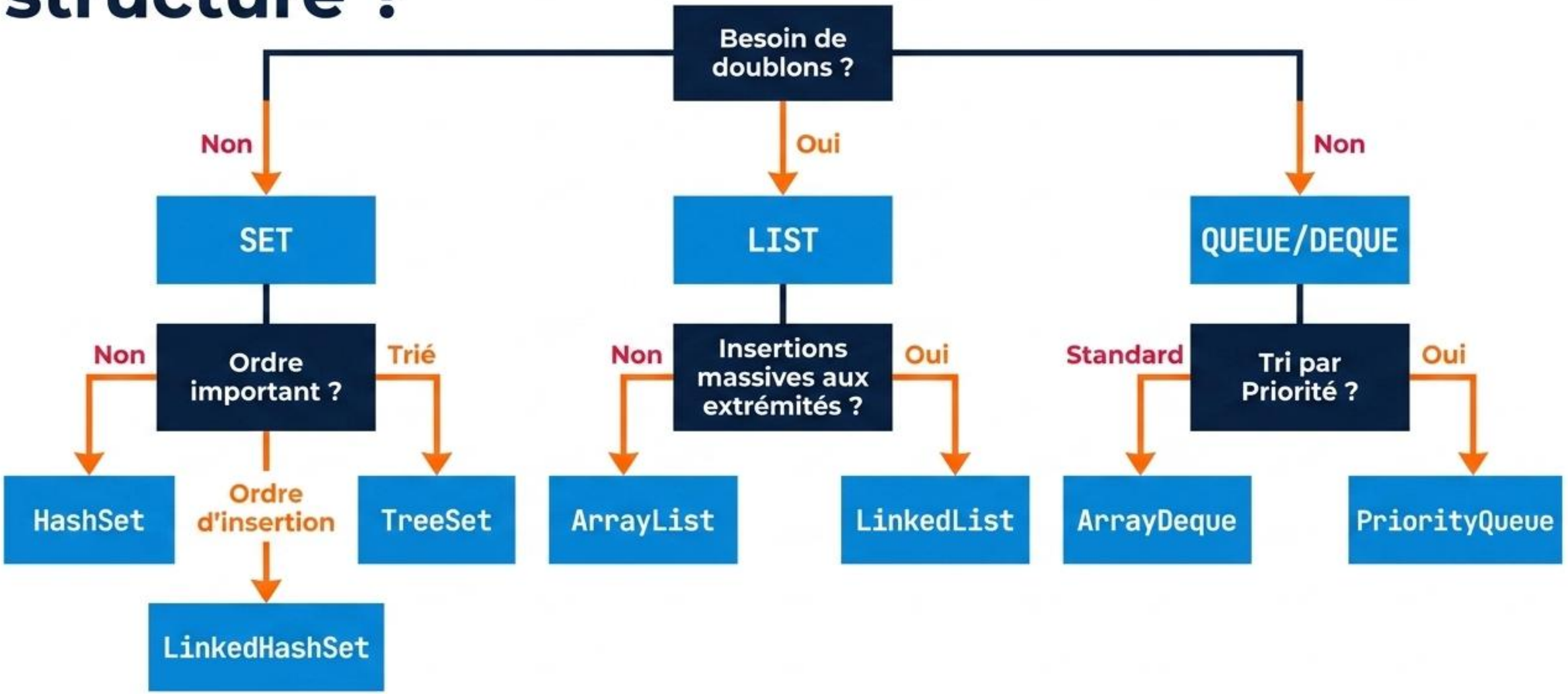
```
Queue<Integer> pq = new PriorityQueue<>();

pq.offer(100);
pq.offer(5);
pq.offer(50);

pq.poll(); ⟶ // Retourne 5 (le plus petit)
```

Règle d'or : Ne plus utiliser la classe obsolète Stack.

Comment choisir la bonne structure ?



Bonnes Pratiques et Pièges Communs

Déclaration générique

Toujours déclarer par l'interface (`List<String> list = new ArrayList<>()`), pas par l'implémentation. Garantit un couplage faible.

Vérification du vide

Toujours utiliser `isEmpty()` au lieu de `size() == 0`. Complexité O(1) garantie, même sur les collections concurrentes.

L'Arbre Intolérant

Insérer un élément `null` dans un `TreeSet` ou une `PriorityQueue` provoque immédiatement une erreur fatale (`NullPointerException`).

ConcurrentModificationException

Ne jamais modifier une collection pendant son parcours avec une boucle for-each. Utilisez `iterator.remove()` ou `removeIf()`.